

[Java] Resource 微服务整体解析



文档维护者：孙家正

此解析基于代码版本为：**#bab5530 (Commit #231)**，编制日期为**2026-04-17**，请注意时效性。

`wisepen-resource-service` 是 WisePen 云平台的资源管理与权限控制微服务。它负责：

- 资源元数据管理：资源的注册、重命名、属性更新、分页列表查询
- 标签体系：基于树形结构的标签/文件夹系统，支持个人空间和小组空间
- 细粒度权限控制：基于位掩码的 ACL（访问控制列表）模型，支持标签级、资源级、用户级三层权限覆盖
- 生命周期管理：软删除 → 回收站 → 定时硬删除 → 通知下游微服务清除物理文件

该服务是平台的资源中枢，其他微服务（如文档服务、笔记服务）通过 Feign 调用其内部接口来注册资源、查询信息和校验权限。

领域模型

核心概念关系

- 一个资源可以同时挂载在多个空间（个人 + 多个小组）下的多个标签上，通过 `groupBinds` 数组管理
- 每个空间拥有独立的标签树，通过 `groupId` 隔离
- 小组资源配置为权限计算提供兜底参数

空间模型

个人空间

- GroupId 约定：`p_{userId}`（如 `p_10086`），由常量 `PERSONAL_GROUP_PREFIX = "p_"` 定义
- 系统节点：首次访问时自动懒创建两个根级 path 节点
 - `/`（ROOT）：所有个人文件的默认挂载点
 - `.Trash`（回收站）：移入此节点或其子孙表示逻辑删除
- 特殊规则：

- 资源在个人空间内必须且只能有一个 `isPath=true` 的标签，且必须位于 `tagIds` 列表首位
- 个人标签不能设置权限配置（`aclGrantMode`、`specifiedUsers`、`grantedActionsMask` 强制置空）
- 资源移入回收站时，自动剥离所有小组绑定和覆盖权限

小组空间

- `GroupId`：小组的数字 ID 字符串（如 `"12345"`）
- 文件组织模式：由 `GroupResConfigEntity.fileOrgLogic` 决定
- 权限体系完整生效：标签级 ACL、资源级覆盖、用户级特权均参与计算

文件组织模式

由枚举 `FileOrganizationLogic` 定义：

模式	说明	标签限制
FOLDER	文件夹模式（默认）	同一小组内每个资源至多挂载一个标签
TAG	标签模式	同一小组内资源可挂载多个标签

权限模型

权限模型是本服务最核心也是最复杂的部分。它基于位掩码实现高效的权限计算和存储。

动作掩码（ResourceAction）

每个权限动作占据一个 bit 位：

动作	位值	十进制	说明
DISCOVER	$1 \ll 0$	1	列表可见（发现权）
VIEW	$1 \ll 1$	2	在线阅读
EDIT	$1 \ll 2$	4	协同编辑
DOWNLOAD_WATERMARK	$1 \ll 3$	8	导出/下载带水印版
DOWNLOAD_ORIGINAL	$1 \ll 4$	16	下载源文件

预定义常量：

- `ALL_ACTIONS = 31` （所有权限）

• `DEFAULT_MEMBER_ACTIONS = DISCOVER | VIEW | DOWNLOAD_WATERMARK = 11`

隐含权限（Implied Mask）：高级权限自动包含低级权限：

- `EDIT` → 隐含 `VIEW` + `DISCOVER`
- `DOWNLOAD_ORIGINAL` → 隐含 `DOWNLOAD_WATERMARK` + `VIEW` + `DISCOVER`
- `DOWNLOAD_WATERMARK` → 隐含 `VIEW` + `DISCOVER`
- `VIEW` → 隐含 `DISCOVER`

ACL 授予模式（AclGrantMode）

控制标签的权限向谁下发：

模式	Code	行为
ALL	0	全员下发配置的掩码
ONLY_ADMIN	1	仅管理员下发掩码，普通成员为 0
WHITELIST	2	仅白名单内用户下发掩码，其他人为 0
BLACKLIST	3	白名单外用户下发掩码，黑名单内用户为 0

标签层级权限继承

权限配置沿标签树从当前节点向根方向查找，采用最近非空优先策略。`resolveTagPermission`方法独立解析两个维度：

1. ACL 授予模式（`aclGrantMode` + `specifiedUsers`）
2. 要授予的动作掩码（`grantedActionsMask`）

两个维度分别向上溯源，各自取最近的非空配置。如果遍历到根节点仍未找到：

- `aclGrantMode` 默认为 `ALL`
- `grantedActionsMask` 默认为 `GroupResConfig.defaultMemberActionsMask`

资源级覆盖与用户特权A

`ResourceItemEntity` 上存在两层覆盖：

字段	类型	作用
<code>overrideGrantedActionsMask</code>	<code>Integer</code>	非空时替代标签策略计算出的掩码
<code>specifiedUsersGrantedActionsMask</code>	<code>Map<String, Integer></code>	按用户 ID 映射独立掩码，最高优先级

小组默认成员掩码

`GroupResConfigEntity.defaultMemberActionsMask`：当标签树上没有任何节点配置 `grantedActionsMask` 时，使用此值作为兜底。默认值为 `DEFAULT_MEMBER_ACTIONS` (`DISCOVER + VIEW + DOWNLOAD_WATERMARK = 11`)。

预计算 ACL (ComputedGroupAcl)

为了避免每次访问资源时都实时遍历标签树，系统在标签/权限变更时异步预计算并持久化到 `ResourceItemEntity.computedGroupAcls` 字段：

每个 `ComputedGroupAcl` 包含：

- `baseMask`：普通成员的基础权限掩码
- `userMasks`：特殊用户的独立权限掩码（白名单/黑名单模式下生成）

编译规则（按 `AclGrantMode`）：

- `ALL` → `baseMask = effectiveMask`
- `ONLY_ADMIN` → `baseMask = 0`
- `WHITELIST` → `baseMask = 0`，白名单用户写入 `userMasks`
- `BLACKLIST` → `baseMask = effectiveMask`，黑名单用户在 `userMasks` 中置 0

权限校验流程

完整优先级链（从高到低）：

- 资源所有者 → `ALL_ACTIONS`（全权限）
- 资源级用户特权 (`specifiedUsersGrantedActionsMask[userId]`)
- 小组管理员/所有者 → `ALL_ACTIONS`（全权限）
- 资源级覆盖 (`overrideGrantedActionsMask`) 前提是用户通过标签策略获得了非零权限
- 标签策略：通过 `resolveTagPermission` 沿祖先链解析 + `AclGrantMode` 过滤

`checkPermission` 返回三元组 `ResourceCheckPermissionResDTO`：

- `resourceAccessRole`：以什么身份访问 (`OWNER / OWNER_SPECIFIED / GROUP_ADMIN / GROUP_MEMBER / NONE`)
- `permissionSources`：权限来源小组 ID 集合
- `allowedActions`：允许执行的动作列表

访问角色 (ResourceAccessRole)

角色	Level	说明

OWNER	4	资源所有者
OWNER_SPECIFIED	3	被所有者定向赋予特权的用户
GROUP_ADMIN	2	小组管理员 / 小组所有者
GROUP_MEMBER	1	通过标签策略获得权限的普通成员
NONE	0	无任何权限

数据存储

集合名	文档类	用途
wisepen_resource_items	ResourceItemEntity	资源主表
wisepen_tags	TagEntity	标签主表
wisepen_group_res_config	GroupResConfigEntity	小组资源配置
wisepen_resource_trash	ResourceItemEntity	资源回收站
wisepen_tags_trash	TagEntity	标签回收站（小组解散时使用）
wisepen_group_res_config_trash	GroupResConfigEntity	小组配置回收站

对外 API（用户侧）

所有用户侧控制器均标注 @CheckLogin，要求请求头中包含合法的 userId。通过 SecurityContextHolder 获取当前登录用户及其小组角色。

资源操作 ResourceItemController

路径前缀： /resource/item

- POST /renameResource：重命名资源。仅资源所有者可操作。
- POST /removeResources：软删除资源（移入资源回收站）。仅资源所有者可操作，逐一断言所有权。
- POST /changeResourceTags：更新资源在某个空间下的标签绑定。权限规则：
 - 不传 groupId：操作个人空间标签，需为资源所有者
 - 传 groupId：小组管理员/所有者可直接操作；普通成员需为资源所有者
- POST /changeResourceActionPermission：修改资源级权限覆盖和用户特权。仅资源所有者可操作。

- GET /listResources：分页列出资源。查询场景：
 - 个人所有资源：不传 groupId 且不传 tagIds；
 - 小组所有资源：传 groupId 且不传 tagIds；
 - 个人指定标签：不传 groupId 且传 tagIds；
 - 小组指定标签：传 groupId 且传 tagIds。

请求参数：

参数	类型	默认值	说明
groupId	String	空	小组 ID，为空时查个人空间
tagIds	List<String>	空	标签 ID 列表
tagQueryLogicMode	QueryLogicEnum	OR	多标签组合逻辑（AND/OR）
resourceType	String	空	资源类型过滤
page	int	1	页码（1-based）
size	int	20	每页条数
sortBy	ResourceSortBy	UPDATE_TIME	排序字段
sortDir	SortDirectionEnum	DESC	排序方向

响应：R<PageResult<ResourceItemResponse>>。其中 ResourceItemResponse 包含资源基本信息 + 当前空间下的标签映射（Map<tagId, tagName>）。

标签操作 ResourceTagController

路径前缀：/resource/tag

权限规则（控制器内 checkPermission 方法）：不传 groupId：自动设为个人空间；传 groupId + 读操作：须为小组成员；传 groupId + 写操作：须为小组 ADMIN 或 OWNER。

- GET /getTagTree：获取完整标签树。个人空间首次访问时自动创建 / 和 .Trash 系统节点。
- POST /addTag：创建新标签节点。业务规则：
 - 禁止在回收站内创建节点
 - 禁止使用 / 或 .Trash 作为名称
 - 同级不能重名
 - 子节点的 isPath 自动继承父节点

- POST /changeTag: 更新标签元信息和权限配置。业务规则:
 - 回收站内节点禁止修改;
 - 系统节点禁止修改;
 - 禁止改名为系统保留名;
 - 个人空间禁止设置权限配置;
 - 权限变更时触发子树下所有资源的 ACL 重算。
- POST /moveTag: 拖拽移动标签 (改变树形层级)。业务规则:
 - 不能移动到自身或其子孙节点下 (防止环形依赖);
 - 不能跨类型移动 (FOLDER和TAG不能互相移动);
 - 目标位置同级不能重名 (回收站除外);
 - 移入回收站时触发 TagTrashedEvent, 剥离子树下所有资源的小组权限;
 - 正常移动时触发 TagChangedEvent, 重算子树下资源的 ACL;
 - 移动会批量重算所有子孙节点的 ancestors 数组。
- POST /removeTag: 级联删除标签及其所有子孙节点。业务规则:
 - 系统节点禁止删除;
 - FOLDER 类型的节点在非回收站内时禁止直接物理删除 (需先移入回收站);
 - 删除后发布 TagDeletedEvent。

小组资源配置 GroupResConfigController

路径前缀: /resource/groupConfig

- GET /getConfig: 获取小组资源配置。须为小组成员。查不到记录时返回默认值 (FOLDER 模式 + 默认成员权限)。
- POST /changeConfig: 创建或更新小组资源配置。须为 ADMIN 或 OWNER。当 defaultMemberActionsMask 发生实质变化时, 会遍历该组所有资源, 逐一发送 Kafka ACL 重算事件。

内部 API (服务间 Feign)

- POST /internal/resource/addRes: 注册/创建资源。自动将资源绑定到所有者个人空间的根目录或指定路径标签。
 - POST /internal/resource/changeResAttr: 更新资源元数据 (名称、预览图、大小等)。按 DTO 非空字段更新, 资源不存在时静默返回。
 - POST /internal/resource/getResourceInfo: 获取资源详情 (含 VIEW 权限校验)。
- 响应: R<ResourceItemResponse>, 包含:

- 资源基本信息 + 所有者展示信息（通过 Feign 调用用户服务获取）
- `currentActions`：当前用户可执行的动作列表
- `currentTags`：用户有权查看的标签映射
- 仅所有者可见：`overrideGrantedActions` 和 `specifiedUsersGrantedActions`
- POST `/internal/resource/checkResPermission`：权限检查（不抛异常，返回权限结构体）。
响应：`R<ResourceCheckPermissionResDTO>`
 - `resourceAccessRole`：访问身份
 - `permissionSources`：权限来源组（Set）
 - `allowedActions`：允许的动作（List）资源不存在时返回 `ResourceAccessRole.NONE`，不抛异常。
- POST `/internal/resource/dissolveGroup`：小组解散级联操作：
 - 软删该组所有标签 → 标签逐条写入 `wisepen_tags_trash`，主表删除
 - 软删该组资源配置 → 设置 `dissolvedAt`，写入 `wisepen_group_res_config_trash`，主表删除

30 天后由 GC 定时任务彻底清理。

服务层业务逻辑（仅包括复杂逻辑）

ResourceServiceImpl

- 资源创建（`createResourceItem`）
 - 拷贝 DTO 属性到 Entity，保存获取 `resourceId`
 - 确定目标路径标签：优先使用 `dto.pathTagId`，否则查找个人空间的 `/` 根节点
 - 构建 `ResourceUpdateTagsRequest` 调用 `updateResourceTags` 绑定标签
 - 失败回滚：如果标签绑定过程抛异常，删除已创建的资源后重新抛出
- 标签绑定更新（`updateResourceTags`）
 - 查找资源 → 不存在抛 `RESOURCE_NOT_FOUND`
 - if `tagIds` 为空：
 - 个人空间禁止清空标签
 - 移除该组的 `GroupTagBind`
 - else：
 - 校验 Tag 存在性和所属空间一致性
 - 个人空间：校验恰好 1 个 `isPath` 标签且在首位

- 检查是否移入回收站 → 是则剥离所有小组绑定和覆盖权限

- 小组空间：FOLDER 模式下校验最多 1 个标签
- 覆盖或新增 GroupTagBind

d. 保存资源

e. ACL 重算事件发布：个人空间仅在移入回收站时发此事件；小组空间无论何时都发。

- ACL 核心计算（calculateResourceGroupAcl），由 Kafka 消费者调用

a. 读取资源绑定记录

b. 遍历每个 GroupTagBind：

- 跳过个人空间绑定
- 取首标签（index 0），查出 TagEntity
- 读取 GroupResConfig 的 defaultMemberActionsMask
- 调用 resolveTagPermission 沿祖先链解析权限
- 应用资源级 overrideGrantedActionsMask（若有）
- 按 AclGrantMode 编译为 ComputedGroupAcl

c. 使用 MongoTemplate.updateFirst 原子写入 computedGroupAcls

- 标签事件响应，通过 `@TransactionalEventListener` 在事务提交后处理：

事件	处理方法	行为
<code>TagChangedEvent</code> t	<code>afterTagNodeChanged</code>	查找受影响的资源，逐一发 Kafka ACL 重算事件
<code>TagDeletedEvent</code> t	<code>afterTagNodeDeleted</code>	Path 标签删除→资源软删；非 Path→移除绑定+发 Kafka
<code>TagTrashedEvent</code> t	<code>stripGroupPermissions</code> n	剥离所有非个人空间的绑定和权限覆盖

- 资源级权限覆盖（updateResourceActionPermission）

a. 查找资源 → 不存在抛 RESOURCE_NOT_FOUND

b. overrideGrantedActions：

- 非空 → actionsToPermissionCode 转掩码存入 overrideGrantedActionsMask
- null → 清除覆盖（回退到标签策略）

c. specifiedUsersGrantedActions：

- 非空 → 逐用户转掩码存入 specifiedUsersGrantedActionsMask
- null → 清除所有用户特权

d. 保存资源

e. 发布 ACL 重算事件

- **权限校验 (checkPermission)** 这是供其他微服务在执行敏感操作（导出 PDF、分享链接等）前调用的鉴权方法。与 `getResourceInfo` 中的内联鉴权不同，它不抛异常，而是返回完整的权限结构体，由调用方自行决策。

调用逻辑：

代码块

```
1  1. 查找资源
2    └ 不存在 → 返回 ResourceAccessRole.NONE (不抛异常)
3  2. 所有者判断
4    └ userId == ownerId → 返回 OWNER + ALL_ACTIONS
5  3. 提取用户定向特权
6    └ userMask = specifiedUsersGrantedActionsMask[userId] (可能为 null)
7  4. 群组上下文判断
8    └ noGroupContext = (用户不在任何组) OR (资源不在任何组)
9    └ 如果 noGroupContext AND userMask == null → 返回 NONE
10 5. 遍历资源绑定的每个 GroupTagBind (群组权限计算):
11   └ 跳过个人空间绑定 (p_ 前缀)
12   └ 用户不在该组 → 跳过
13   └ 用户是该组 ADMIN/OWNER → 立即返回 GROUP_ADMIN + ALL_ACTIONS
14   └ 普通成员路径:
15       └ 取首标签 → resolveTagPermission 沿祖先链解析
16       └ 按 AclGrantMode 判断用户是否有资格获得掩码:
17         └ ALL → 有资格
18         └ WHITELIST 且用户在列表中 → 有资格
19         └ BLACKLIST 且用户不在列表中 → 有资格
20         └ 其他 → 无资格, 跳过该组
21   └ 有资格时:
22       └ 身份提升为 GROUP_MEMBER
23       └ 记录权限来源组
24       └ actionsMask |= resolved.grantedActionsMask (多组按位或累加)
25 6. 应用资源级覆盖 (三层优先级):
26   └ 如果用户在某组内获得了非零权限:
27       └ overrideGrantedActionsMask 非空 → 替代 actionsMask
28   └ 如果 userMask 非空 (定向用户特权, 最高优先级):
29       └ 身份改为 OWNER_SPECIFIED
30       └ actionsMask = userMask
31       └ 清空 permissionSources
32 7. 返回三元组: { resourceAccessRole, permissionSources, allowedActions }
```

- 与 `calculateResourceGroupAcl` 的关键区别：`checkPermission` 是实时计算，不依赖预计算的 `computedGroupAcls`，而是现场调用 `resolveTagPermission` 走标签祖先链。这保证了在 ACL 重算存在延迟的情况下，鉴权结果仍然基于最新的标签配置。
- 资源详情获取与内联鉴权 (`getResourceInfo`) `getResourceInfo` 内部嵌入了一套独立的鉴权逻辑，与 `checkPermission` 的区别是：

- 它使用预计算的 `computedGroupAcls` 而非实时溯源，换取更快的响应速度
- 它只检查 `VIEW` 权限，不满足则直接抛异常
- 它同时返回资源详情 + 权限信息

调用逻辑：

代码块

```
1  1. 查找资源 → 不存在抛 RESOURCE_NOT_FOUND
2  2. 计算 currentActionsMask:
3      |— userId == ownerId → ALL_ACTIONS
4      |— 否则，按优先级链:
5          |— (a) specifiedUsersGrantedActionsMask[userId] 非空 → 使用该值
6          |— (b) 遍历 computedGroupAcls:
7              |— 用户不在该组 → 跳过
8              |— 用户是 ADMIN/OWNER → ALL_ACTIONS, break
9              |— 普通成员:
10                 |— mask = userMasks[userId] ?? baseMask
11                 |— currentActionsMask |= mask (多组累加)
12          |— (c) 如果最终 mask 非零且 overrideGrantedActionsMask 非空:
13              |— currentActionsMask = overrideGrantedActionsMask
14  3. VIEW 权限检查
15      |— !hasAction(currentActionsMask, VIEW) → 抛 RESOURCE_PERMISSION_DENIED
16  4. 组装响应:
17      |— Feign 调用 RemoteUserService 获取 ownerInfo (失败降级)
18      |— currentActions = 当前用户可执行的动作列表
19      |— currentTags = 仅返回用户有权访问的组的标签映射
20      |— 仅所有者可见: overrideGrantedActions + specifiedUsersGrantedActions (解包掩码为枚举列表)
```

资源列表查询 (`listResources`)

a. 预处理：

- 未传 `groupId` → 自动设为 `"p_{userId}"` (个人空间)
- 传了 `groupId` → `assertInGroup` 校验用户必须是小组成员，获得 `userGroupRole`

b. 个人空间回收站排除 (小组无回收站)：

- 查出该个人空间 `parentId="0"`, `tagName=".Trash"` 的回收站根节点

- 查出回收站下所有子孙标签
- 收集回收站根 + 所有子孙的 tagId → excludeTrashIds 黑名单

仅在未指定 tagIds 时生效（用户主动浏览回收站时不拦截）

c. 核心分页查询：

- 回收站黑名单排除；
- 资源类型过滤；
- 个人空间所有权隔离：个人空间的安全隔离完全依赖 `ownerId == userId`，不走 ACL；
- 标签绑定匹配：
 - AND 模式 → 资源必须同时挂载所有指定标签
 - OR 模式 → 资源挂载了任一指定标签即可
- ACL 可见性过滤（仅小组非管理员成员）

代码块

```
1  criteria: OR(  
2    // 情况 A：自己的资源（免检）  
3    ownerId == userId,  
4  
5    // 情况 B：资源级定向用户特权  
6    specifiedUsersGrantedActionsMask.{userId} 的 DISCOVER 位为 1,  
7  
8    // 情况 C：预计算 ACL 中该用户有专属掩码  
9    computedGroupAcls.{groupId}.userMasks.{userId} 的 DISCOVER 位为 1,  
10  
11   // 情况 D：用户无专属掩码，走基础掩码  
12   AND(  
13     computedGroupAcls.{groupId}.userMasks.{userId} 不存在,  
14     computedGroupAcls.{groupId}.baseMask 的 DISCOVER 位为 1  
15   )  
16 )
```

情况 C 和 D 是互斥的——如果用户在 `userMasks` 中有条目（无论值是多少），就用该条目的值；只有当条目不存在时才回退到 `baseMask`。这正是 `BLACKLIST` 模式工作的原理：黑名单用户在 `userMasks` 中被写入了 `0`，所以即使 `baseMask` 包含 `DISCOVER`，他们也无法通过情况 D 的条件。

- d. 批量获取当前页所有资源在当前组下的 tagIds
- e. 汇总所有 tagId → 批量查询 TagRepository 获取 tagId → tagName 映射
- f. 拼装 ResourceItemResponse 列表 + PageResult

- 软删与硬删流程

软删除 (softRemoveResources) :

- a. 批量查出资源实体
- b. 逐条设置 deletedAt = now(), 写入 wisepen_resource_trash
- c. 从主集合 deleteAllById

硬删除 (hardRemoveResources) :

- a. 从 wisepen_resource_trash 查出目标资源
- b. 从 trash 集合 remove
- c. 发送 资源物理删除事件 通知下游

- 回收站权限剥离 (stripGroupPermission) 当资源所属的标签被移入个人回收站时触发:

- a. 按 trashedTagIds 查出所有受影响的资源
- b. 对每个资源:
 - 移除所有非个人空间的 groupBinds
 - 清空 overrideGrantedActionsMask
 - 清空 specifiedUsersGrantedActionsMask
 - 清空 computedGroupAcls

3. 批量保存

4. 逐条发 ACL 重算事件

TagServiceImpl

- 标签树获取 (getTagTree)

- 一次性查出该组所有节点 (避免 N+1)
- 个人空间首次访问时自动创建 / 和 .Trash
- 在内存中递归组装树结构 (buildTree 方法)

- 标签移动 (moveTag)

- 禁止向回收站内部节点移动;
- 不能移到自身;
- 系统节点禁止移动;
- 目标位置同级重名校验 (回收站例外) ;
- 防环形依赖: 新父不能是自身的子孙;
- 跨类型检查: isPath 必须一致;

- 更新当前节点 `parentId + ancestors`;
- 批量重算所有子孙节点的 `ancestors`;
- 事件发布：如果移入回收站则发布 `TagTrashedEvent`；否则则发布 `TagChangedEvent`。
- 小组解散标签处理
 - `softRemoveAllTagByGroupId`：逐条写入 `wisepen_tags_trash`，然后从主表按 `groupId` 批量删除
 - `hardRemoveAllTagByGroupId`：从 `trash` 集合查出后发布 `TagDeletedEvent`，再物理删除

GroupResServiceImpl

- 配置读取 (`getGroupResConfig`)
 - 查不到记录时返回默认值：`FOLDER` 模式 + `DEFAULT_MEMBER_ACTIONS` (11)。
- 配置更新 (`upsertGroupResConfig`)
 - 懒创建：查不到时 new 一个默认实体
 - 检测 `defaultMemberActionsMask` 是否实质变化
 - 变化时：查出该组所有资源，逐一推送 Kafka ACL 重算事件

Kafka 消息

ACL 重算（服务内消费）

- Topic: `wisepen-resource-acl-recalc-topic`
 - 生产者: `KafkaEventPublisherImpl.publishAclRecalculateEvent`
 - 消息体: `AclRecalculateMessage { resourceId, triggerSource }`
 - Key: `resourceId` (保证同一资源的事件有序)
 - 触发时机: 标签变更、标签删除、资源标签修改、资源权限覆盖修改、小组默认掩码变更、资源移入回收站
 - 消费者: `AclRecalculateConsumer`
 - Consumer Group: `wisepen-resource-acl-recalc-group`
 - 逻辑: 调用 `resourceService.calculateResourceGroupAcl(resourceId)`

资源物理删除通知（跨服务消费）

- Topic: `wisepen-resource-physical-destroy-topic`
 - 生产者: `KafkaEventPublisherImpl.publishResDeletedEvent`

- 消息体：`ResourceDeletedMessage { typedResourceIds: Map<ResourceType, List<String>> }`
- 触发时机：GC 定时任务硬删资源后
 - 消费者：下游文件存储等微服务（不在本服务内）

定时任务

ResourceGcTask

GC 三阶段：

阶段 1：清理过期小组配置（`cleanupExpiredGroups`）

1. 从 `wisepen_group_res_config_trash` 查出 `dissolvedAt < threshold` 的记录
2. 对每条记录：
 - 硬删该组标签（`tagService.hardRemoveAllTagByGroupId` → 发布 `TagDeletedEvent`）
 - 硬删该组配置（从 `trash` 集合 `remove`）

阶段 2：清理个人回收站中的过期项（`cleanupExpiredResourcesInUserTrash`）

3. 查出所有个人空间的 `.Trash` 根节点
4. 对每个回收站：
 - 查出直属子文件夹中 `updateTime < threshold` 的，逐个强制删除（级联删除 `Tag` + 资源）
 - 查出直接孤立在 `.Trash` 下 `updateTime < threshold` 的资源，调 `softRemoveResources` 移入审计集合

阶段 3：清理审计集合中的过期资源（`cleanupExpiredResources`）

5. 从 `wisepen_resource_trash` 查出 `deletedAt < threshold` 的记录
6. 调 `hardRemoveResources` → 从 `trash` 集合物理删除 + 发 `Kafka` 通知下游